■# ADR-001: Convert JSON schema validation step from agent to deterministic code

Status

Accepted

Context

The orchestration includes a step that validates each handoff document against schemas/handoff.json before the next subagent consumes it. The step-classification document marks it a strong candidate on all four signals: stable harness scores, identical output on repeated runs, a specification a developer can implement directly, and a run on every workflow invocation. The classification also names the edge case the deterministic version must preserve: a required field that is present but blank must be treated as invalid, not accepted just because the key exists.

Decision

Replace the agentic validation step with a small script that checks the handoff document against schemas/handoff.json and rejects it with a clear error if any required field is missing, null, or empty.

Alternatives considered

- Keep the agentic step. Rejected. The calibration log before-conversion entry shows this step averaged 45 seconds of cycle time and $0.003 in token cost per run across three runs, while passing all 7 deterministic harness checks. It contributes no language understanding the task requires.
- Convert to deterministic code. Accepted after integration. The after-conversion entry shows the script averaged 0.2 seconds and $0 token cost across three runs, produced identical output on all three, and passed the same 7 deterministic checks with no regression.
- Shrink the agent with a tighter prompt and a smaller model. Rejected. It lowers token cost but keeps inference latency, output variance, and governance overhead.

Consequences

Cycle time fell from 45 seconds to 0.2 seconds, token cost fell from $0.003 per run to zero, and output variance fell to zero. Known trade-off: the script returns a terse structured result rather than a generated prose explanation. If the structured problem list proves too thin for debugging, enrich the deterministic messages before considering a return to an agent.

Evidence

- Classification: `docs/step-classification.md`, JSON schema validation marked strong candidate.
- Before-conversion measurement: `docs/calibration-log.md`, before-conversion entry.
- After-conversion measurement in isolation: `docs/calibration-log.md`, after-conversion entry.
- Integrated end-to-end regression check: `docs/calibration-log.md`, integrated end-to-end regression check.

Open Risks

- The deterministic validator may need updates if schemas/handoff.json changes.
- A schema-valid handoff can still be semantically incorrect; structural validation does not replace agent or human judgment.
- Error messages may need to become more descriptive if future debugging requires more context.
- New edge cases must be added to deterministic tests before the validator is considered safe under an expanded handoff format.

ADR-002: Convert path-based routing step from agent to deterministic code

Status

Proposed

Context

The workflow contains a routing step that assigns work to a role based on changed file paths. The decision is structural: files under tests/ go to the tester, files under docs/ go to documentation/project-management work, and source changes go to the implementer. The main edge case is an unmatched path.

Decision

Replace the agentic routing step with scripts/route_task_deterministic.py, which applies explicit ordered path rules and defaults unmatched paths to implementer.

Alternatives considered

- Keep the agentic router. Rejected for the sample because the behavior is a lookup table, not judgment.
- Convert to deterministic code. Preferred, pending integration and full measurement.
- Keep both router and agent as fallback. Rejected because that preserves cost and governance complexity for a structural operation.

Related decisions

Builds on ADR-001. Two lessons from that conversion shaped this one: identify the input class an agent handled by judgment that a script might miss, and write a test for it; mark model-only rubric dimensions as not applicable rather than failures.

Evidence

Pending measurement and integration.

ADR-003: Rubric Design for Agentic Evaluation

Status

Accepted

Context

The agentic engineering pipeline needs an evaluation method that can judge output quality consistently across different workflow runs.

Deterministic checks are useful for structural requirements such as schema validity, routing, policy enforcement, and required files, but they cannot fully evaluate qualitative properties such as whether an answer is correct, sufficiently grounded in retrieved evidence, focused on the requested task, or clear enough for a human reviewer to act on.

The capstone therefore requires a rubric-based evaluation layer that complements deterministic checks rather than replacing them.

The evaluation design also needs to be simple enough to apply repeatedly during calibration and regression testing.

Decision

Use a four-dimension rubric with a 1â€“4 scoring scale.

The rubric dimensions are:

1\. Correctness — whether the final output performs the requested task without significant errors or omissions.

2\. Task Adherence — whether the workflow stays within the requested scope instead of performing unrelated work.

3\. Groundedness — whether claims based on retrieved information accurately reflect the supporting sources.

4\. Clarity — whether the output is organized and actionable without unnecessary rework.

Each dimension has a minimum passing score of:

text

3
```
The overall passing threshold is:


```text

12
```

This means a run must meet the acceptable threshold across all four dimensions rather than compensating for a weak dimension with a high score elsewhere.

The rubric is stored in:

text

eval/rubric.json
```
Each dimension contains:


* a description;

* a pass threshold;

* scoring definitions for levels 1â€“4;

* examples illustrating weak and strong outcomes.


## Rejected Alternatives


### Alternative 1: Use only deterministic pass/fail tests


Rejected because deterministic checks can verify structure and policy rules but cannot reliably judge qualitative output such as reasoning qual


A workflow could pass every structural test while still producing an unclear or poorly grounded result.


### Alternative 2: Use one overall quality score


Rejected because a single score would hide the reason a run failed.
```
```

For example, a response could be technically correct but drift outside the task, or it could be clear but poorly grounded.

Separating the rubric into dimensions makes failures easier to diagnose and gives calibration work a specific target.

Alternative 3: Use a larger or more complex rubric

Rejected because additional dimensions would increase scoring overhead without clear evidence that more categories were necessary for this work

The current four dimensions cover the major qualitative risks while remaining practical for repeated evaluation.

Evidence

The rubric is implemented in `eval/rubric.json` with four dimensions:

* correctness;
* task adherence;
* groundedness;
* clarity.

Each dimension requires a score of at least 3, and the overall passing threshold is 12.

Calibration evidence also shows that the wider evaluation system is used alongside deterministic and policy checks rather than in isolation.

The calibration log records a successful integrated regression check with:

```text
24/24 harness checks passing
```

and the policy suite passing.

The same calibration evidence records real workflow near-misses involving:

* excessive implementer permissions;
* reviewer skill overreach;
* retrieval beyond a role's classification ceiling.

These failures demonstrate why evaluation must measure more than whether a task merely produced output.

Consequences

Positive consequences

* Evaluation failures can be traced to a specific quality dimension.
* Calibration changes can target the relevant weakness instead of changing the entire workflow.
* The rubric complements deterministic validation and policy tests.
* A score of 3 provides a clear minimum acceptable quality level.
* Explicit examples make scoring more repeatable.
* Groundedness ensures retrieval quality is reflected in final outputs rather than measured only at the retrieval layer.

Negative consequences

* Rubric scoring may require an agent or human evaluator.
* Qualitative scoring can still contain some judgment variability.
* The rubric increases evaluation cost compared with deterministic checks alone.
* Examples may need to evolve if the workflow changes significantly.

Open Risks

1\. Rubric-based scoring may vary slightly across evaluator models or repeated scoring runs.

2\. The current rubric may not capture specialized reviewer or tester concerns if those roles require more detailed future evaluation.

3\. A model-based judge could incorrectly score an output even when deterministic evidence suggests a different conclusion.

4\. Changes to the workflow may require new examples or additional dimensions.

These risks are mitigated by keeping deterministic checks as the first evaluation layer and treating rubric scoring as a complementary qualitative layer rather than the sole source of truth.

Decision Summary

The pipeline will use a four-dimension, 1â€“4 rubric with a minimum score of 3 per dimension and an overall passing threshold of 12.

This design was selected because it provides enough qualitative detail to diagnose failures while remaining simple enough to use repeatedly during calibration and regression testing.

ADR-004: Persistent Memory Layout and Scope

Status

Accepted

Context

The multi-agent engineering workflow needs persistent state and reusable reference knowledge across runs, but memory must not become an unrestricted shared context that leaks information between roles or sessions.

The repository already contains a persistent memory area under:

text

.memory/
```
  Current artifacts include:


  * `storage.db`

  * `storage-audit.log`

  * `.memory/reference/`


  The reference directory contains reusable project and lesson knowledge such as:


  * API and spreadsheet references

  * cost information

  * CSV format decisions

  * error codes

  * feature notes

  * security guidance

  * governance lessons

  * memory-scope guidance


  The design therefore needs to distinguish persistent project state from reusable reference material and to control which roles are allowed to m


  ## Decision


  Use a layered memory layout with persistent project state separated from reference knowledge.


  ### Persistent project state


  The SQLite database:
```

```text
.memory/storage.db
```

stores persistent workflow or project state.

Changes to persistent storage are traceable through:

text

.memory/storage-audit.log

```
### Reference knowledge


Reusable reference documents are stored under:


```text
.memory/reference/
```

These files contain project decisions, feature notes, security guidance, lessons learned, and other context that may be retrieved when a role requires it.

Role-scoped mounting

Persistent memory is not mounted for every role.

Roles that require stateful implementation context may receive memory access, while read-only or advisory roles can run without the persistent memory mount.

For example, the current governance design records the Implementer with:

text

workspace: read-write

memory: mounted

```
while the Reviewer and Tester operate with memory omitted.


This reduces unnecessary exposure of persistent state.


### Separation principle


The memory design separates:


1\. **Persistent project state**

2\. **Reusable reference knowledge**

3\. **Temporary role context**

4\. **Agent instructions and skills**


Persistent memory is reserved for information that should survive across runs. Temporary task-specific context should remain in the current wor


## Rejected Alternatives


### Alternative 1: Give every agent the same persistent memory mount


Rejected because not every role requires persistent state.


A Reviewer or Tester only needs enough context to inspect and report. Giving all roles access to the same persistent memory would increase the


### Alternative 2: Store all knowledge directly in the agent prompt


Rejected because prompts are not a good durable store for evolving project knowledge.
Embedding all prior decisions and reference material into prompts would increase context size, make updates harder to track, and reduce separat
```
```

Alternative 3: Store everything in one undifferentiated database

Rejected because reference documents and mutable workflow state serve different purposes.

Keeping reusable references in a distinct directory makes them easier to inspect, version, retrieve, and reason about separately from mutable c

Alternative 4: Use only session-local memory

Rejected because the workflow needs continuity across runs.

Project decisions, lessons, and reusable reference material would otherwise have to be reconstructed repeatedly.

Evidence

The current repository contains:

```text
.memory/storage.db
.memory/storage-audit.log
.memory/reference/
```

The reference directory includes multiple durable knowledge artifacts, including governance, retrieval, security, feature, and memory-scope lessons.

The repository also contains:

text

.memory/reference/lesson-memory-scope-check.md

which documents memory-scope concerns.

The agent-run script includes logic for mounting persistent memory selectively rather than treating it as universally available.

Governance documentation also differentiates role-level memory behavior. The Implementer may run with memory mounted, while read-only roles suc

This supports the principle that persistent memory should be scoped to role responsibility rather than globally exposed.

Consequences

Positive consequences

* Persistent state survives across workflow runs.
* Reusable reference knowledge remains inspectable and separate from mutable state.
* Audit logging improves traceability of persistent-memory operations.
* Role-specific mounting reduces unnecessary memory exposure.
* Temporary task context does not automatically become long-term project memory.
* The design supports least privilege.

Negative consequences

* The architecture is more complex than using one shared memory store.

* Developers must decide whether new information belongs in persistent state, reference knowledge, or temporary context.
* Reference content can become stale if it is not reviewed.
* Persistent memory must be sanitized before public submission.

Open Risks

1\. Reference documents may become outdated and continue to influence retrieval.
2\. Incorrect role configuration could mount persistent memory for a role that should not receive it.
3\. Sensitive information could accidentally be written into persistent storage.
4\. Audit logs themselves may contain operational information that requires sanitization.
5\. SQLite storage is sufficient for the current capstone scale but may not be appropriate for larger concurrent production workloads.

These risks are mitigated through role-scoped access, governance policy, audit evidence, retrieval validation, and final sanitization review.

Decision Summary

The capstone uses a layered persistent-memory design.

Mutable workflow state is stored in `.memory/storage.db`, persistent operations are recorded in `.memory/storage-audit.log`, and reusable knowl

Memory is not globally mounted. Access is scoped according to role responsibility, supporting least privilege and reducing cross-role context l

ADR-005: MCP Boundaries and Least-Privilege Tool Access

Status

Accepted

Context

The multi-agent pipeline requires persistent storage and retrieval capabilities, but exposing every MCP operation to every agent would violate

The repository implements two MCP-backed capabilities:

* persistent storage;
* retrieval.

These capabilities serve different purposes and have different risk profiles.

Storage operations can modify or remove project state, while retrieval can expose information at different data-classification levels.

The architecture therefore needs explicit boundaries that define:

* which roles can call which operations;
* which roles may modify state;

```
* which roles may inspect audit information;

* which roles may retrieve data;

* what classification level each role may access.


## Decision


Use separate MCP servers for storage and retrieval, with explicit role-based allow-lists enforced at the operation level.


### Storage MCP


The storage server is located under:


```text

mcp-servers/storage/
```

The storage allow-list grants operations by role.

Current permissions are:

```
| Operation | Allowed Roles |
| ---------- | ---------- |
| `write_entry` | implementer, orchestrator |
| `read_entry` | implementer, reviewer, tester, project-manager, orchestrator, documentation-writer |
| `list_entries` | implementer, reviewer, tester, project-manager, orchestrator, documentation-writer |
| `update_entry` | implementer, orchestrator |
| `delete_entry` | orchestrator |
| `audit_read` | orchestrator |
```

This means destructive and audit-sensitive operations are restricted more heavily than read operations.

Retrieval MCP

The retrieval server is located under:

text

mcp-servers/retrieval/

```
Retrieval permission is granted per role and includes a data-classification ceiling.


Current configuration is:


| Role | Retrieval | Classification Ceiling |
| ---------- | ---------- | ---------- |
| implementer | granted | internal |
| reviewer | granted | internal |
| tester | granted | internal |
| orchestrator | granted | confidential |
| documentation-writer | granted | internal |
| project-manager | denied | not applicable |


The Project Manager is explicitly denied retrieval because its current workflow responsibility does not require it.


### Schema boundary


Agent handoffs are also constrained by:
```text

schemas/handoff.json
```

The schema requires:

* task_id

* summary

* owner

Each required field must be a non-empty string.

This gives the workflow a structural boundary before downstream agent execution continues.

Rejected Alternatives

Alternative 1: Expose all MCP operations to every role

Rejected because most roles do not need full storage or retrieval authority.

For example, a Reviewer needs to inspect state but should not be able to modify or delete it.

The calibration log also records a near-miss involving excessive delete access for the Implementer, demonstrating that over-broad grants create a real governance risk.

Alternative 2: Use one shared global MCP permission set

Rejected because storage and retrieval have different security concerns.

Storage controls state mutation, while retrieval controls information exposure.

Combining them into one undifferentiated permission model would make it harder to reason about risk and apply least privilege.

Alternative 3: Rely only on agent prompt instructions

Rejected because prompt-level rules are not sufficient enforcement.

An agent may misunderstand, ignore, or drift beyond instructions.

Operation-level allow-lists provide an independent enforcement boundary.

Alternative 4: Allow retrieval without classification ceilings

Rejected because successful tool authorization alone does not mean every role should see every document.

The calibration log records an Implementer request for confidential material that exceeded its internal ceiling.

Classification ceilings therefore protect data exposure independently from basic tool access.

Evidence

The current repository contains:

text

mcp-servers/storage/allow-list.json

mcp-servers/retrieval/allow-list.json

mcp-servers/storage/schema.json

mcp-servers/retrieval/schema.json

schemas/handoff.json
```
  The storage allow-list shows that only the Orchestrator may:


  ```text

  delete_entry

  audit_read
```
The Implementer may write and update state but may not delete it.

Reviewer, Tester, Project Manager, and Documentation Writer are limited to read-oriented storage access.

The retrieval allow-list records classification ceilings of:

text

internal
```
  for Implementer, Reviewer, Tester, and Documentation Writer, and:


  ```text

  confidential
```
for the Orchestrator.

The Project Manager is explicitly denied retrieval.

The calibration log provides supporting evidence for these boundaries through near-misses involving:

* over-broad Implementer delete access;

* Reviewer tool overreach;

* retrieval above a role's classification ceiling.

These events show that the allow-list design is responding to observed risks rather than hypothetical ones.

Consequences

Positive consequences

* Tool access follows least privilege.

* Destructive storage operations are tightly restricted.

* Audit information is limited to the coordinating role.

* Retrieval exposure is constrained by classification level.

* Role responsibilities remain easier to reason about.

* Policy violations can be tested independently of agent prompts.

* MCP boundaries are visible in version-controlled configuration.

Negative consequences

* Adding a new role requires explicit updates to allow-lists.

* Permission configuration must remain synchronized with governance documentation.

* Incorrect allow-list changes could unintentionally widen or block access.

* Classification rules introduce additional configuration complexity.

Open Risks

1\. A future role may require a permission that is not currently represented.

2\. Governance documentation and allow-list configuration could drift apart.

3\. A classification label could be assigned incorrectly to retrieved content.

4\. Schema validation currently permits additional properties, which may allow fields beyond the minimum required handoff structure.

5\. MCP server implementation bugs could weaken enforcement even when configuration is correct.

These risks are mitigated through CI checks, policy tests, calibration review, schema validation, red-team testing, and required justification for permission widening.

Decision Summary

The capstone uses separate MCP storage and retrieval boundaries with explicit role-based allow-lists.

Storage permissions are granted by operation, retrieval permissions are constrained by role and classification ceiling, and handoffs are structurally validated before downstream execution.

This design keeps MCP capabilities narrow, auditable, and aligned with least-privilege governance.

ADR-006: Subagent Scoping and Routing

Status

Accepted

Context

The engineering workflow contains multiple responsibilities that should not be handled by one unrestricted agent.

Planning, implementation, review, testing, project communication, and workflow coordination require different capabilities, levels of autonomy, and tool access.

Using one general-purpose agent for all steps would make it difficult to enforce least privilege and would increase the chance that one role could accidentally perform work that belongs to another role.

The pipeline therefore requires explicit subagent boundaries and a routing model that assigns work according to task type.

Decision

Use an Orchestrator to coordinate specialized subagents.

Current specialized roles include:

* Implementer

* Reviewer

* Tester

* Project Manager

* Documentation Writer

The Orchestrator remains responsible for parent-workflow coordination and delegation.

The current orchestration path is:

text

Orchestrator
  |
v
Detect Changed Files
  |
v
Validate Handoff
  |
v
Route Task
  |
+--> Implementer
  |
+--> Reviewer
  |
+--> Tester
  |
+--> Project Manager

```
The routed outputs then continue through policy and evaluation gates.


## Role Scoping


### Implementer


The Implementer owns code and implementation work.


It may modify project state within its permitted scope and receives the tools needed for implementation.


It does not receive destructive storage access such as `delete_entry`.


### Reviewer


The Reviewer owns code-review work.


It is intentionally advisory and read-only.


The Reviewer is not permitted to modify stored project state or apply implementation changes.


### Tester


The Tester owns test execution and test reporting.


Its workspace remains read-only for tracked source files, preventing test execution from becoming an implementation path.
### Project Manager
The Project Manager owns description and summary work.
```

It may read project state but does not receive implementation, testing, or retrieval capabilities that are unnecessary for its role.

Documentation Writer

The Documentation Writer may read stored state and retrieve internal reference material when documentation work requires context.

Its access remains narrower than the Orchestrator.

Routing Rules

The routing design maps work to the role that owns the task.

Current route categories include:

Route	Role
Code or implementation	Implementer
Code review	Reviewer
Tests	Tester
Description or project summary	Project Manager

The routing layer is deliberately kept separate from agent execution.

This allows routing behavior to be inspected, tested, and eventually converted to deterministic logic when the categories become stable enough.

Deterministic Routing Direction

The repository includes a deterministic routing example as decision-history evidence.

This reflects the right-tool principle:

If routing depends only on stable, precisely defined structural conditions, it should not remain agentic indefinitely.

Agentic routing is appropriate only when task classification still requires ambiguity handling or contextual interpretation.

Rejected Alternatives

Alternative 1: Use one agent for the entire workflow

Rejected because one agent would require broad permissions across implementation, review, testing, storage, retrieval, and project communicatio

This would weaken least privilege and make audit trails less meaningful.

Alternative 2: Give every subagent the same tools
Rejected because tool needs differ by responsibility.
For example:

```
* the Reviewer should not modify state;
* the Tester should not become an implementation role;
* the Project Manager does not need retrieval;
* the Orchestrator requires broader coordination and audit access.

### Alternative 3: Route tasks manually every time

Rejected because manual routing would reduce automation value and would not scale well as the workflow repeats.

Human checkpoints should remain for consequential decisions, not routine role assignment when the route can be determined safely.

### Alternative 4: Make routing fully deterministic immediately

Rejected because routing should only be deterministic when task categories and edge cases are sufficiently stable and specifiable.

The project preserves a deterministic routing example but does not assume every future routing decision can be reduced to rules.

## Evidence

The orchestration documentation shows:

```text
Orchestrator
-> Detect changed files
-> Validate handoff
-> Route task
-> Specialized role
-> Policy and eval gates
```

The routing-and-tool-grant map records distinct route conditions for:

* Implementer
* Reviewer
* Tester
* Project Manager

The governance policy applies different permissions, classification ceilings, skills, autonomy levels, and container permissions to each role.

Calibration evidence also supports role separation.

Observed near-misses included:

* an Implementer with overly broad delete access;
* a Reviewer nearly triggering a testing skill;
* an Implementer requesting data above its classification ceiling.

These near-misses demonstrate why role scope must be enforced rather than treated as documentation only.

Consequences

Positive consequences

* Each agent has a clear responsibility.
* Tool grants can remain narrow.
* Review independence is preserved.
* Testing authority is separated from implementation authority.
```

* Routing and execution can be tested independently.

* Future deterministic conversion of stable routing rules remains possible.

* Audit evidence becomes easier to interpret by role.

Negative consequences

* More roles increase configuration and maintenance overhead.

* Routing mistakes can send a task to the wrong specialist.

* Changes to workflow responsibilities may require synchronized updates across agent definitions, governance policy, allow-lists, and tests.

* Some tasks may cross role boundaries and require escalation or multiple agents.

Open Risks

1\. Ambiguous tasks may be routed incorrectly.

2\. A new workflow category may not map cleanly to an existing role.

3\. Role definitions and routing rules could drift apart over time.

4\. Overlapping responsibilities could produce duplicate or conflicting outputs.

5\. Deterministic routing could become too rigid if converted before edge cases are understood.

These risks are mitigated through evaluation, regression testing, governance checks, calibration, and human escalation when task ownership is unclear.

Decision Summary

The capstone uses an Orchestrator plus scoped subagents rather than one unrestricted agent.

Routing assigns work to specialized roles according to task type, and each role receives only the tools, data access, and autonomy required for its responsibility.

This design improves least privilege, auditability, review independence, and future right-tool conversion opportunities.

ADR-007: Least-Privilege Governance Policy

Status

Accepted

Context

The multi-agent engineering pipeline allows AI agents to interact with project state, retrieval systems, tools, skills, and repository resources.

Without explicit governance, an agent could receive broader access than its task requires, modify information it should only inspect, retrieve data above its intended classification level, or activate tools outside its assigned responsibility.

Calibration identified concrete near-miss patterns, including:

1\. An Implementer nearly receiving destructive delete_entry access.

2\. A Reviewer nearly triggering the run-tests skill despite being a read-only advisory role.

3\. An Implementer requesting a document classified above its internal retrieval ceiling.

These observations showed that role expectations written only in prompts were not sufficient. Governance needed to be explicit, versioned, testable, and enforced independently of agent behavior.

Decision

Adopt a deny-by-default, least-privilege governance policy for every agent role.

The governing principle is:

> Every role starts with no access. Every permission must be explicitly granted and justified. Anything not explicitly granted is denied.

The policy is version-controlled in:

text

docs/governance-policy.md

```
  Governance applies across multiple dimensions:


  * MCP server and operation access

  * Skill activation

  * Data-classification ceilings

  * Agent autonomy

  * Human checkpoints

  * Container permissions
```

* Persistent-memory access
* Role-specific state mutation permissions

Role-Based Governance

Permissions attach to stable roles rather than temporary agent instances.

This allows governance rules to remain consistent across repeated workflow executions.

Orchestrator

The Orchestrator receives broader coordination privileges because it owns workflow-level orchestration and auditing.

It may access destructive storage and audit operations that are denied to narrower subagents.

Implementer

The Implementer may modify approved project state but does not receive destructive delete authority.

Its retrieval ceiling is `internal`.

Higher-risk actions outside its normal implementation scope require a human checkpoint.

Reviewer

The Reviewer is advisory and read-only.

It may inspect relevant state and retrieve internal context but may not modify stored project state or activate implementation-oriented tools.

Tester

The Tester may execute approved testing responsibilities and inspect required context but may not modify tracked project state.

Project Manager

The Project Manager receives only the access required for project descriptions and summaries.

Retrieval is denied because the current workflow does not require it.

Documentation Writer

The Documentation Writer receives read-oriented storage access and internal retrieval access for documentation tasks.

It does not inherit the broader authority of the Orchestrator or Implementer.
Permission Widening
Permissions may not be expanded silently.

To widen access, the governance policy requires a pull request containing:

1\. the proposed permission;

2\. a concrete justification;

3\. confirmation that the new permission does not conflict with a known calibration near-miss.

This prevents gradual access creep and creates reviewable evidence for governance changes.

Human Checkpoints

Human review is required when an action exceeds the safe autonomy assigned to a role.

Examples include:

* an Implementer attempting shell activity outside the approved testing path;
* writes outside the intended feature scope;
* a Reviewer attempting to perform state-changing work;
* a Tester attempting to modify tracked repository state;
* a request for permissions beyond the role's approved policy.

The system should escalate these cases rather than automatically broadening agent authority.

Enforcement

Governance is not implemented only as written guidance.

The repository includes enforcement mechanisms and evidence such as:

```text
mcp-servers/storage/allow-list.json
mcp-servers/retrieval/allow-list.json
eval/test_policy.py
eval/test_governed_files.py
eval/enforcement-verification.md
eval/red-team-prompts.md
eval/red-team-results.md
.github/workflows/ci.yml
```

These artifacts allow policy assumptions to be checked automatically and tested against attempted bypasses.

Rejected Alternatives

Alternative 1: Trust agent prompts to enforce permissions

Rejected because prompts describe expected behavior but do not provide an independent authorization boundary.

A model may misunderstand or drift beyond instructions.

Tool-level enforcement is therefore required.

Alternative 2: Give every agent broad permissions for convenience

Rejected because convenience does not justify unnecessary authority.

An error by a broadly privileged agent would have a larger blast radius.

The Implementer delete-access near-miss provides direct evidence of this risk.

Alternative 3: Use identical permissions for every role

Rejected because roles have different responsibilities.

A Reviewer does not need write access, a Project Manager does not currently need retrieval, and only the Orchestrator requires audit and destructive storage authority.

Alternative 4: Allow agents to request and automatically receive additional permissions

Rejected because an agent should not be able to authorize its own privilege escalation.

Permission widening requires explicit review and justification.

Alternative 5: Treat governance as a one-time configuration

Rejected because agent roles, tools, workflow requirements, and failure patterns can evolve.

Governance must therefore remain versioned, testable, and reviewable.

Evidence

The governance policy records least privilege as the default rule.

The storage allow-list restricts:

text

delete_entry

audit_read

```
  to the Orchestrator.


  The retrieval allow-list enforces classification ceilings, including:


  * `internal` for Implementer, Reviewer, Tester, and Documentation Writer;

  * `confidential` for Orchestrator;

  * retrieval denied for Project Manager.


  The calibration log records three concrete near-misses that informed these boundaries.


  The repository also includes policy tests, governed-file tests, enforcement verification, red-team prompts and results, and CI/CD integration.


  Together, these artifacts show that governance decisions are connected to observed risks and executable controls.


  ## Consequences


  ### Positive consequences


  * Agent permissions remain narrow and explainable.

  * The blast radius of an agent failure is reduced.

  * Review and testing roles preserve independence.

  * Sensitive retrieval is limited by classification level.

  * Permission changes leave an auditable decision trail.

  * Governance violations can be tested in CI.

  * Human accountability remains available for higher-risk decisions.


  ### Negative consequences
  * Governance configuration requires ongoing maintenance.

  * New roles or tools require explicit policy updates.
```

* Legitimate work may occasionally be blocked until permissions are reviewed.

* Multiple policy artifacts must remain synchronized.

Open Risks

1\. Policy documentation and executable allow-lists could drift apart.

2\. A future tool could introduce capabilities not covered by the current policy.

3\. A classification label could be incorrect or incomplete.

4\. Branch-protection or CI settings could be weakened outside the repository files.

5\. Authorized users with bypass privileges may still circumvent required pull-request or status-check workflows.

The final risk is particularly important because repository push output has shown that protected-branch rules can be bypassed by an authorized

These risks are mitigated through policy tests, pipeline-integrity checks, red-team exercises, explicit access reviews, human checkpoints, and

Decision Summary

The capstone adopts deny-by-default, role-based, least-privilege governance.

Permissions are explicit, scoped to role responsibilities, constrained by data classification and container boundaries, and backed by executabl

Privilege widening requires review rather than autonomous agent approval.

This design provides enforceable boundaries around agent behavior while preserving human accountability for higher-risk actions.

ADR-008: Prepare Tester acceptance-criteria verification for deterministic conversion

Status

Proposed

Context

The Tester runs after the Reviewer passes an implementation and before the workflow can continue to the Project Manager. It receives the accept

Iteration Log Run 2 (2026-08-10) shows that this step provides more than mechanical test execution. The Tester identified that several acceptan

The mechanical portions of this step, such as running tests and reporting passed, failed, and skipped counts, appear specifiable. However, the

Decision

Keep the complete Tester acceptance-criteria verification step agentic for now while treating its mechanical test-running and result-counting p

Alternatives Considered
Alternative 1: Convert the complete Tester step to deterministic code now
Rejected for now. The available evidence does not demonstrate stability across at least two full calibration cycles or identical decisions acro

Alternative 2: Keep the entire Tester step agentic

Viable for now, because interpretation of natural-language acceptance criteria still requires judgment. However, keeping purely mechanical oper

Alternative 3: Convert only the mechanical portions and retain agent judgment for coverage verification

Preferred future direction, pending evidence. Test execution and passed/failed/skipped result collection could potentially become deterministic

Consequences

The immediate consequence is that no implementation changes will be made as part of this exercise. The Tester remains responsible for acceptanc

A future partial conversion could reduce inference work and make test execution and result counting more predictable and easier to audit. The t

The known behavior that must be preserved is the ability to detect when a test suite has no failing tests but still leaves an acceptance criter

Evidence

- `docs/iteration-log.md`, Run 2 - Full Product Review workflow, dated 2026-08-10: the Tester identified missing automated coverage for localSt
- `agents/tester.md`: defines the Tester's inputs, read-only tool boundary, responsibility to compare test results against each acceptance crit
- `docs/step-classification.md`, Tester acceptance-criteria verification entry, committed in `59914f9`.
- Before-and-after conversion measurements do not exist yet because this ADR is Proposed and this exercise stops before implementation.